

Chương này trình bày quá trình phân tích thiết kế, hiện thực và đánh giá hệ thống đặt vé và quản lý sự kiện được phát triển trong đề án. Trước hết, chương làm rõ kiến trúc phần mềm được lựa chọn và cách áp dụng kiến trúc đó vào hệ thống thực tế. Tiếp theo, chương mô tả thiết kế tổng quan, thiết kế chi tiết, quá trình xây dựng ứng dụng, kết quả đạt được, kiểm thử và định hướng triển khai. Các nội dung trong chương này đóng vai trò kết nối giữa phần khảo sát yêu cầu ở các chương trước với phần đóng góp và tổng kết ở các chương sau, qua đó cho thấy hệ thống đã được tổ chức và hiện thực hóa như thế nào trên cả phía ứng dụng di động lẫn phía máy chủ.

0.1 Thiết kế kiến trúc

0.1.1 Lựa chọn kiến trúc phần mềm

Đối với bài toán đặt vé và quản lý sự kiện trên thiết bị di động, hệ thống cần đồng thời hỗ trợ nhiều chức năng khác nhau như quản lý tài khoản, tìm kiếm sự kiện, quản lý sự kiện, đặt vé, thanh toán, nhắn tin, thông báo và check-in bằng mã mã phản hồi nhanh (Quick Response - QR). Các chức năng này có liên hệ chặt chẽ với nhau, dùng chung dữ liệu và cùng phục vụ một luồng nghiệp vụ xuyên suốt từ lúc người dùng khám phá sự kiện cho tới lúc tham dự thực tế. Vì vậy, kiến trúc phù hợp nhất cho hệ thống trong phạm vi đề án là kiến trúc client-server theo hướng mobile-first, kết hợp với tổ chức phần mềm theo mô hình ba lớp ở phía máy chủ. Theo cách hiểu này, ứng dụng di động đóng vai trò lớp giao diện và tương tác với người dùng, máy chủ backend đóng vai trò lớp xử lý nghiệp vụ, còn cơ sở dữ liệu là lớp lưu trữ dữ liệu tập trung.

Kiến trúc ba lớp được lựa chọn thay cho các hướng như microservice hay backendless vì nó phù hợp hơn với quy mô hiện tại của hệ thống. Nếu sử dụng microservice, mỗi nhóm chức năng như tài khoản, sự kiện, vé, thông báo hay chat có thể được tách thành các dịch vụ độc lập. Tuy nhiên, với phạm vi của đề án, cách tiếp cận đó làm tăng độ phức tạp triển khai, quản lý giao tiếp giữa các dịch vụ và đồng bộ dữ liệu, trong khi lượng người dùng và số lượng giao dịch chưa ở mức cần phân tán mạnh. Ngược lại, kiến trúc đơn khối có tổ chức theo lớp giúp hệ thống dễ xây dựng, dễ bảo trì hơn trong giai đoạn phát triển đầu tiên, đồng thời vẫn đủ rõ ràng để mở rộng khi cần. So với mô hình backendless, việc xây dựng backend riêng còn cho phép kiểm soát chặt chẽ hơn các nghiệp vụ đặc thù như quản lý nhiều hạng vé, kiểm soát trạng thái thanh toán, gửi lời mời, kiểm tra vé đã dùng hay chưa và xác nhận check-in theo sự kiện cụ thể.

Xét trên hệ thống cụ thể của đề tài, lớp giao diện và tương tác được hiện thực bằng ứng dụng di động React Native kết hợp Expo. Thành phần này bao gồm các

màn hình chức năng trong thư mục `frontend/pages`, các thành phần giao diện dùng lại trong `frontend/components`, hệ điều hướng trong `frontend/-navigators`, cùng các ngữ cảnh quản lý trạng thái như xác thực và bản địa hóa trong `frontend/context`. Nhiệm vụ của lớp này là tiếp nhận thao tác của người dùng, tổ chức luồng di chuyển giữa các màn hình, hiển thị dữ liệu và gửi yêu cầu đến backend thông qua các service như `auth.service`, `event.service`, `ticket.service`, `notification.service` hay `chat.service`. Nói cách khác, đây là phần gần nhất với người dùng cuối và chịu trách nhiệm thể hiện toàn bộ chức năng của hệ thống trên thiết bị di động.

Lớp xử lý nghiệp vụ được tổ chức ở phía backend theo một dạng MVC điều chỉnh cho ứng dụng API và dịch vụ thời gian thực. Trong hệ thống này, vai trò tương đương với phần Model được thể hiện bởi các schema và model Mongoose trong thư mục `backend/src/models`, nơi quản lý cấu trúc dữ liệu của người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và message. Vai trò tương đương với phần Controller là các controller trong `backend/src/controllers`, phụ trách xử lý từng luồng nghiệp vụ như đăng nhập, tạo sự kiện, đặt vé, xác nhận thanh toán, gửi lời mời, nhắn tin hay check-in. Bổ sung cho đó là các middleware và service trong `backend/src/middleware`, `backend/src/services` và `backend/src/utisls`, dùng để giải quyết những nghiệp vụ dùng chung như xác thực JWT, gửi email, gửi thông báo đẩy, sinh mã QR, gọi ý nội dung sự kiện bằng AI hoặc lập lịch nhắc sự kiện. Phần View trong mô hình MVC truyền thống không được đặt ở phía backend như các ứng dụng web server-rendered, mà được chuyển sang ứng dụng di động ở frontend. Đây là điểm điều chỉnh quan trọng để phù hợp với kiến trúc ứng dụng di động hiện đại.

Ở mức tổ chức truy cập, các route trong `backend/src/routes` đóng vai trò cổng vào của lớp dịch vụ. Mỗi nhóm route như `/api/auth`, `/api/users`, `/api/events`, `/api/tickets`, `/api/notifications` hay `/api/chat` tiếp nhận yêu cầu từ ứng dụng khách, sau đó chuyển tiếp cho controller tương ứng. Điều này tạo nên một chuỗi xử lý tương đối rõ ràng: người dùng thao tác trên giao diện di động, service ở frontend gửi yêu cầu HTTP hoặc sự kiện socket tới backend, route tiếp nhận yêu cầu, controller xử lý nghiệp vụ, model truy cập dữ liệu, sau đó kết quả được trả ngược về giao diện. Với các chức năng gần thời gian thực như chat, hệ thống mở rộng thêm một kênh giao tiếp bằng Socket.IO ở tệp `backend/src/socket.ts`. Đây có thể xem là một thành phần bổ sung cho lớp xử lý nghiệp vụ, giúp hệ thống vượt ra ngoài mô hình request-response thuần túy mà vẫn không phá vỡ cấu trúc tổng thể.

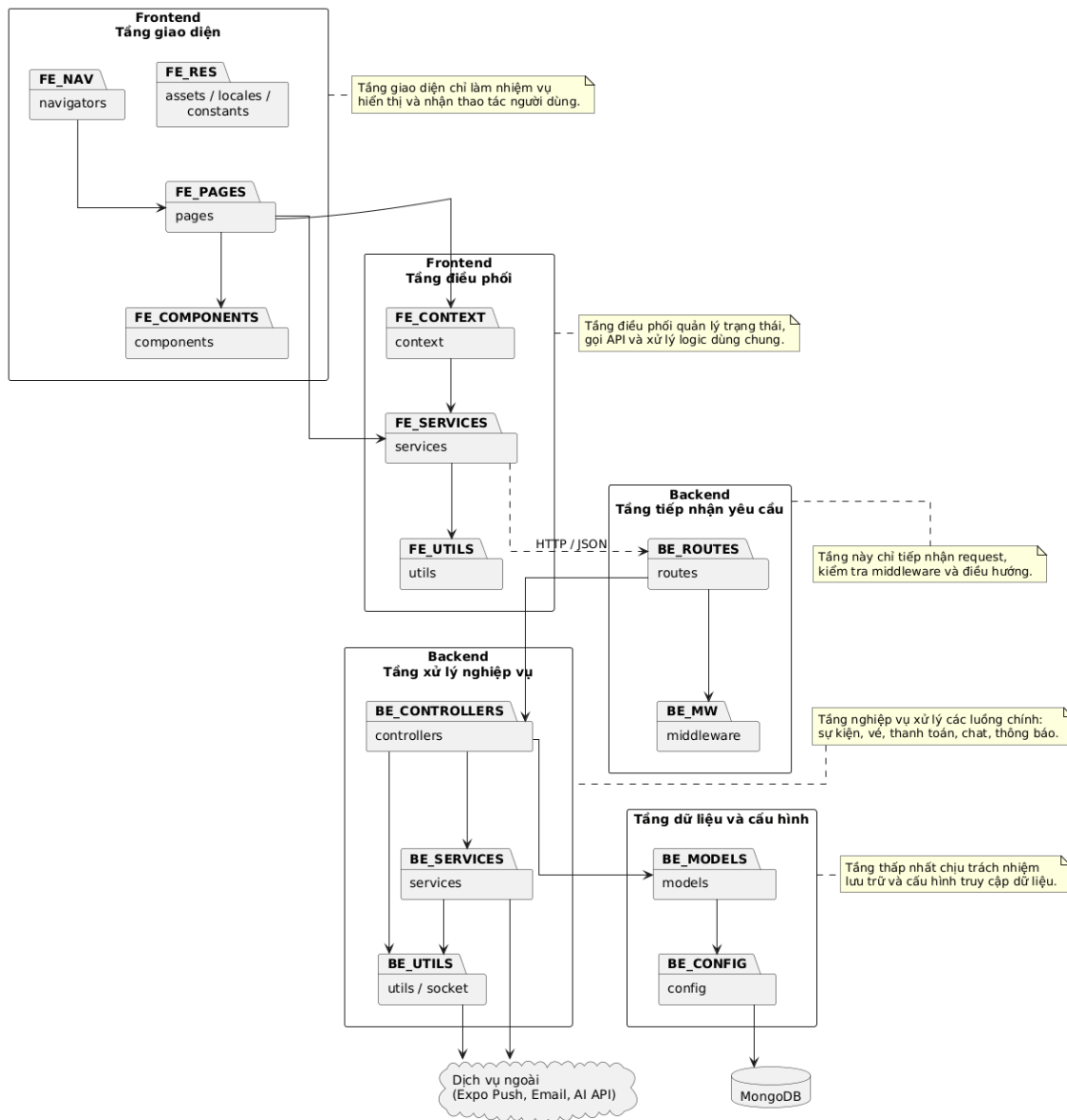
Lớp dữ liệu của hệ thống được xây dựng trên MongoDB và được truy cập thông

qua Mongoose. Việc dùng cơ sở dữ liệu tài liệu giúp hệ thống linh hoạt trong việc biểu diễn các thực thể có cấu trúc thay đổi theo nghiệp vụ, chẳng hạn sự kiện có thể có nhiều hạng vé, trạng thái duyệt, thông tin tọa độ hoặc dữ liệu liên quan đến người tổ chức. Với bài toán của đề tài, dữ liệu không chỉ cần được lưu trữ mà còn phải phản ánh quan hệ giữa nhiều thực thể như người dùng – sự kiện – vé – thanh toán – thông báo. Bởi vậy, lớp dữ liệu không chỉ đóng vai trò lưu trữ mà còn là nền tảng để bảo đảm tính nhất quán cho các thao tác nghiệp vụ quan trọng như tăng số lượng vé đã bán, đánh dấu vé đã check-in hoặc ghi nhận lời mời đã gửi.

Từ góc nhìn tổng thể, kiến trúc được áp dụng cho hệ thống có thể mô tả là kiến trúc client-server ba lớp, trong đó backend được tổ chức theo tinh thần MVC điều chỉnh cho ứng dụng API. Đây là lựa chọn phù hợp với sản phẩm của đề án vì vừa bảo đảm tính mạch lạc trong thiết kế, vừa đủ linh hoạt để tích hợp thêm các thành phần mở rộng như thông báo đẩy, chat thời gian thực, AI hỗ trợ nội dung và kiểm soát tham dự bằng mã QR. Quan trọng hơn, kiến trúc này cho phép nhóm chức năng của hệ thống được phân chia tương đối rõ ràng, giúp cho việc phát triển, bảo trì và mở rộng về sau thuận lợi hơn so với một cấu trúc mã nguồn rời rạc hoặc quá phức tạp so với quy mô hiện tại của đề tài.

0.1.2 Thiết kế tổng quan

Biểu đồ gói tổng quan của hệ thống được xây dựng theo hướng phân tầng rõ ràng giữa phía ứng dụng di động và phía máy chủ. Mục tiêu của cách tổ chức này là giúp các thành phần đảm nhiệm đúng trách nhiệm của mình, hạn chế sự phụ thuộc chéo và thuận lợi cho việc bảo trì sau này. Ở mức khái quát, hệ thống được chia thành các nhóm gói chính gồm: nhóm giao diện người dùng ở frontend, nhóm điều phối và truy cập dịch vụ ở frontend, nhóm tiếp nhận yêu cầu ở backend, nhóm xử lý nghiệp vụ và tích hợp ở backend, cùng với nhóm dữ liệu và cấu hình ở tầng thấp nhất.



Hình 0.1: Biểu đồ gói tổng quan của hệ thống event-booking-app

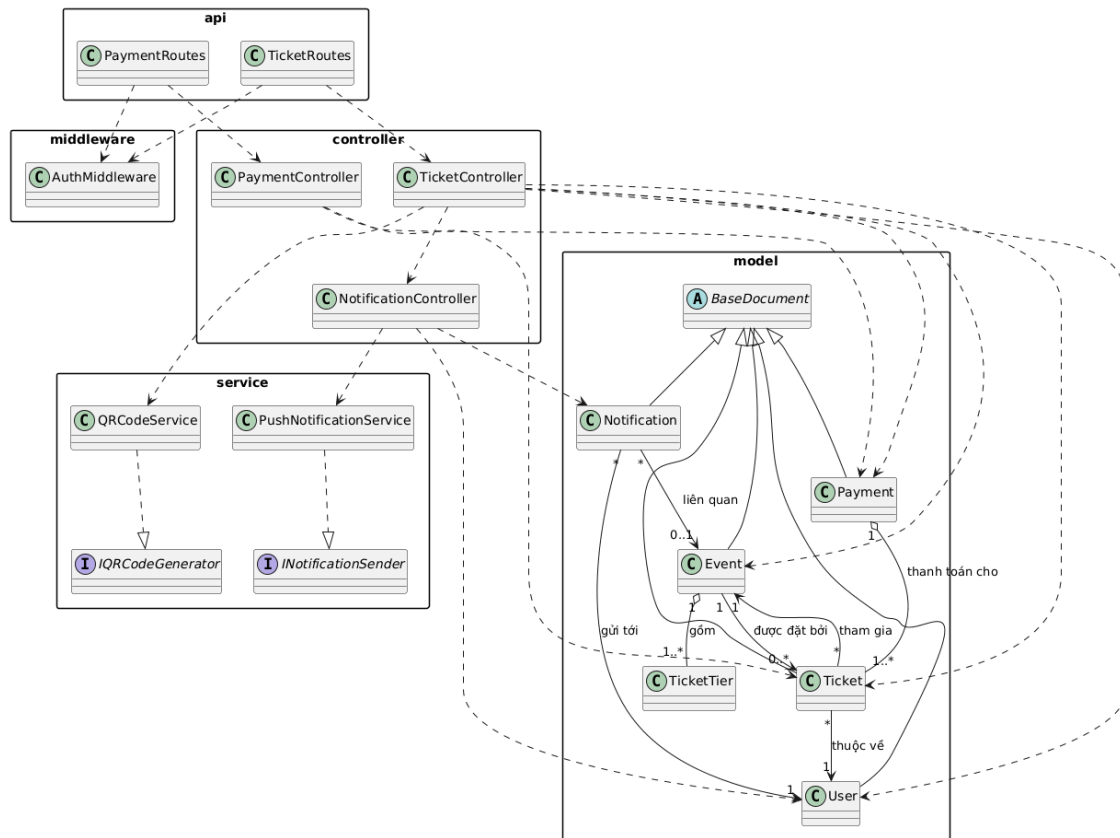
Ở phía frontend, các package có thể được chia thành hai tầng chính. Tầng giao diện bao gồm pages, components, navigators, assets, locales và constants. Trong đó, pages chứa các màn hình nghiệp vụ cụ thể như đăng ký, đặt vé, quản lý sự kiện, chat và thông báo; components chứa các thành phần giao diện dùng lại; navigators chịu trách nhiệm tổ chức luồng điều hướng; còn constants, assets và locales cung cấp các tài nguyên dùng chung. Tầng điều phối ở frontend bao gồm context, services và utils. Package context lưu trạng thái dùng chung như thông tin xác thực và ngôn ngữ, package services thực hiện giao tiếp với backend thông qua API, còn utils xử lý các hàm phụ trợ. Cấu trúc này đảm bảo giao diện không truy cập trực tiếp tới backend hay dữ liệu lưu trữ, mà luôn đi qua tầng service và context tương ứng.

Ở phía backend, package routes là lớp tiếp nhận yêu cầu từ bên ngoài và phân

phối yêu cầu đến các controller tương ứng. Package `middleware` cung cấp các cơ chế kiểm tra xác thực, phân quyền và xử lý lỗi dùng chung cho toàn hệ thống. Package `controllers` là trung tâm điều phối nghiệp vụ, nơi hiện thực các luồng như đăng nhập, tạo sự kiện, đặt vé, gửi lời mời, check-in và xử lý thông báo. Package `services` và `utils` bổ sung các chức năng hỗ trợ hoặc tích hợp như sinh gợi ý nội dung sự kiện bằng AI, gửi email, gửi thông báo đẩy, lập lịch nhắc sự kiện, sinh mã QR và xử lý socket thời gian thực. Package `models` đại diện cho tầng dữ liệu nghiệp vụ, mô tả các thực thể như người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và tin nhắn. Cuối cùng, package `config` chịu trách nhiệm kết nối cơ sở dữ liệu và nạp biến môi trường, tạo nền tảng kỹ thuật cho toàn bộ các package phía trên.

0.1.3 Thiết kế chi tiết gói

Để làm rõ cách tổ chức các lớp trong hệ thống, đề tài lựa chọn nhóm package giải quyết nghiệp vụ đặt vé, thanh toán và tham dự sự kiện để mô tả chi tiết. Đây là nhóm package quan trọng vì nó kết nối trực tiếp nhiều thực thể nghiệp vụ nhất của hệ thống, đồng thời bao trùm các thao tác cốt lõi như chọn vé, xử lý thanh toán, sinh vé điện tử, gửi thông báo và check-in bằng mã QR. Ở mức thiết kế gói, biểu đồ tập trung vào năm nhóm thành phần chính là `api`, `middleware`, `controller`, `service` và `model`. Mỗi nhóm được phân công một vai trò rõ ràng nhằm tránh chồng chéo trách nhiệm và giúp luồng xử lý nghiệp vụ có thể theo dõi được từ đầu tới cuối.



Hình 0.2: Biểu đồ thiết kế gói cho nghiệp vụ đặt vé, thanh toán và tham dự sự kiện

Trong thiết kế này, package `api` gồm các lớp như `TicketRoutes` và `PaymentRoutes`, chịu trách nhiệm tiếp nhận yêu cầu từ phía ứng dụng khách và chuyển yêu cầu sang tầng điều phối. Package `middleware` chứa lớp `AuthMiddleware`, được dùng để kiểm tra token xác thực và bảo vệ các chức năng chỉ dành cho người dùng đã đăng nhập. Package `controller` bao gồm các lớp như `TicketController`, `PaymentController` và `NotificationController`. Đây là nơi tổ chức luồng nghiệp vụ chính, chẳng hạn kiểm tra loại vé, khởi tạo giao dịch, xác nhận thanh toán, gọi dịch vụ sinh mã QR, cập nhật trạng thái vé và gửi thông báo sau giao dịch. Package `service` bao gồm các dịch vụ hỗ trợ như `QRCodeService` và `PushNotificationService`, cùng với các interface tương ứng như `IQRCodeGenerator` và `INotificationSender`. Việc đưa interface vào thiết kế giúp tách biệt giữa phần hợp đồng chức năng và phần hiện thực cụ thể, thuận lợi cho việc thay thế hoặc mở rộng dịch vụ về sau. Package `model` bao gồm các lớp dữ liệu chính như `Event`, `Ticket`, `Payment`, `Notification`, `User` và `TicketTier`, cùng lớp cơ sở trừu tượng `BaseDocument`.

Các quan hệ giữa các lớp trong biểu đồ gói thể hiện rõ cách vận hành của hệ thống. Quan hệ phụ thuộc (dependency) xuất hiện khi các lớp route sử dụng mid-

dleware và controller, hoặc khi controller cần gọi service và truy cập model để xử lý nghiệp vụ. Quan hệ thực thi (implementation) xuất hiện ở chỗ QRCodeService hiện thực IQRCodeGenerator và PushNotificationService hiện thực INotificationSender. Quan hệ kế thừa (inheritance) được dùng để mô tả việc các model nghiệp vụ kế thừa từ lớp cơ sở BaseDocument. Quan hệ kết tập (aggregation) có thể thấy trong trường hợp Payment tập hợp nhiều đối tượng Ticket thuộc cùng một giao dịch, trong khi quan hệ hợp thành (composition) thể hiện ở việc Event bao gồm nhiều TicketTier như một phần cấu thành của sự kiện. Ngoài ra, quan hệ kết hợp (association) được dùng để phản ánh các mối liên hệ nghiệp vụ như mỗi Ticket gắn với một User và một Event, hoặc một Notification được gửi tới một User và có thể liên quan tới một Event cụ thể.

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Phần này có độ dài từ hai đến ba trang. Sinh viên đặc tả thông tin về màn hình mà ứng dụng của mình hướng tới, bao gồm độ phân giải màn hình, kích thước màn hình, số lượng màu sắc hỗ trợ, v.v. Tiếp đến, sinh viên đưa ra các thống nhất/chuẩn hóa của mình khi thiết kế giao diện như thiết kế nút, điều khiển, vị trí hiển thị thông điệp phản hồi, phối màu, v.v. Sau cùng sinh viên đưa ra một số hình ảnh minh họa thiết kế giao diện cho các chức năng quan trọng nhất. Lưu ý, sinh viên không nhầm lẫn giao diện thiết kế với giao diện của sản phẩm sau cùng.

0.2.2 Thiết kế lớp

Phần này có độ dài từ ba đến bốn trang. Sinh viên trình bày thiết kế chi tiết các thuộc tính và phương thức cho một số lớp chủ đạo/quan trọng nhất của ứng dụng (từ 2-4 lớp). Thiết kế chi tiết cho các lớp khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục.

Để minh họa thiết kế lớp, sinh viên thiết kế luồng truyền thông điệp giữa các đối tượng tham gia cho 2 đến 3 use case quan trọng nào đó bằng biểu đồ trình tự (hoặc biểu đồ giao tiếp).

0.2.3 Thiết kế cơ sở dữ liệu

Phần này có độ dài từ hai đến bốn trang. Sinh viên thiết kế, vẽ và giải thích biểu đồ thực thể liên kết (E-R diagram). Từ đó, sinh viên thiết kế cơ sở dữ liệu tùy theo hệ quản trị cơ sở dữ liệu mà mình sử dụng (SQL, NoSQL, Firebase, v.v.)

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Sinh viên liệt kê các công cụ, ngôn ngữ lập trình, API, thư viện, IDE, công cụ kiểm thử, v.v. mà mình sử dụng để phát triển ứng dụng. Mỗi công cụ phải được chỉ rõ phiên bản sử dụng. SV nên kẻ bảng mô tả tương tự như Bảng 1. Nếu có nhiều nội dung trình bày, sinh viên cần xoay ngang bảng.

Mục đích	Công cụ	Địa chỉ URL
IDE lập trình	Eclipse Oxygen a64 bit	http://www.eclipse.org/
v.v.	v.v.	v.v.

Bảng 1: Danh sách thư viện và công cụ sử dụng

0.3.2 Kết quả đạt được

Sinh viên trước tiên mô tả kết quả đạt được của mình là gì, ví dụ như các sản phẩm được đóng gói là gì, bao gồm những thành phần nào, ý nghĩa, vai trò?

Sinh viên cần thống kê các thông tin về ứng dụng của mình như: số dòng code, số lớp, số gói, dung lượng toàn bộ mã nguồn, dung lượng của từng sản phẩm đóng gói, v.v. Tương tự như phần liệt kê về công cụ sử dụng, sinh viên cũng nên dùng bảng để mô tả phần thông tin thống kê này.

0.3.3 Minh họa các chức năng chính

Sinh viên lựa chọn và đưa ra màn hình cho các chức năng chính, quan trọng, và thú vị nhất. Mỗi giao diện cần phải có lời giải thích ngắn gọn. Khi giải thích, sinh viên có thể kết hợp với các chú thích ở trong hình ảnh giao diện.

0.4 Kiểm thử

Phần này có độ dài từ hai đến ba trang. Sinh viên thiết kế các trường hợp kiểm thử cho hai đến ba chức năng quan trọng nhất. Sinh viên cần chỉ rõ các kỹ thuật kiểm thử đã sử dụng. Chi tiết các trường hợp kiểm thử khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục. Sinh viên sau cùng tổng kết về số lượng các trường hợp kiểm thử và kết quả kiểm thử. Sinh viên cần phân tích lý do nếu kết quả kiểm thử không đạt.

0.5 Triển khai

Sinh viên trình bày mô hình và/hoặc cách thức triển khai thử nghiệm/thực tế. Ứng dụng của sinh viên được triển khai trên server/thiết bị gì, cấu hình như thế nào. Kết quả triển khai thử nghiệm nếu có (số lượng người dùng, số lượng truy cập, thời gian phản hồi, phản hồi người dùng, khả năng chịu tải, các thống kê, v.v.)